



Review article

A prefetching indexing scheme for in-memory database systems[☆]Qian Zhang^{*}, Haoyun Song, Kaiyan Zhou, Jianhao Wei, Chuqiao Xiao

Software Engineering Institute, East China Normal University, 200062, Shanghai, China

ARTICLE INFO

Keywords:

Main memory
Prefetching
Database systems
Multi-versioning
Indexing scheme

ABSTRACT

In-memory databases (IMDBs) store all working data in the main memory, making memory access the dominant factor in system performance. Moreover, for modern multi-version systems, the extended version chain makes the access pattern more complex, putting extra pressure on indexing. Our micro-architectural profiling results of existing IMDB indexing schemes show that over half of the execution time goes to memory stalls caused by pointer chasing operations.

This paper proposes a prefetching indexing scheme for modern in-memory database systems. This scheme achieves high performance in the presence of serial accesses in pointer chasing. The essential idea is to use a path prefetcher and a jump pointer prefetcher to hide cache miss latencies induced by indexing searches on versioned tuples. Specifically, this scheme works by associating a read counter with each block and updating the counters in the search keys' access paths. To generate the optimal paths for future prefetches, we present how to continuously evolve the frequent access paths by analyzing those access patterns. Also, we create a jump pointer for each tuple in the leaf nodes to prefetch the head of its version chain. We present a jump pointer queue to accelerate linear version traversal. We achieve the high update performance because our improved search speed more than offsets any increase in prefetching overhead. Evaluations show that the prefetching indexing scheme outperforms the state-of-the-art indexing scheme by up to 70%.

1. Introduction

In the last decade, with the DRAM density increasing and the price per gigabyte decreasing, commodity servers with hundreds of gigabytes of DRAM have been prevailing. Many in-memory database systems (IMDBs) [1–10] have been developed by enterprises and academic institutions to meet the in-time data analysis and decision-making needs in many important applications [11–18]. IMDBs could provide high throughput rates and low latency. It is because, in such a system, the whole database resides in random access memory.

Most IMDBs employ a multi-version concurrency control (MVCC) scheme to achieve efficient transaction management to maximize parallelism without sacrificing serializability [3,5,7–10,17,19,20]. Under MVCC, updates never block reads. An update operation produces a new version of the updated data item and invalidates the predecessor version. All versions of the tuple form a pointer-based version chain (i.e., linked list). Read operations simply use a timestamp to read the visible version of the tuple at the time the query starts.

Also, to reduce update pressure in indexes, these database systems have employed latch-free indirection arrays. The database system uses a

fixed identifier that does not change for each tuple in its index entry. It uses an indirection layer to map the identifier to the head of the tuple's version chain. This approach avoids the problem of having to update the table index to point to a new physical address whenever a tuple is modified. Only the mapping entry needs to change each time. Most of all, a single compare-and-swap operation suffices to install a new version of the tuple without the use of heavy-weight locking methods.

Generally speaking, a root-to-leaf traversal is always needed to search for a key in the tree-like indexes; the nodes in the traversal path are visited sequentially. Moreover, the indirection layer, in combination with the prolonged version chain, exercises extra pressure on the index searches. The current indexing scheme heavily relies on pointer-intensive data structures, i.e., trees and linked lists; pointer chasing is a fundamental behavior [21,22]. While pointers ease the dynamic inserts/updates/deletes in IMDBs, pointer chasing can be inefficient within the CPU, as it introduces several sources of performance degradation [23–29]. For example, irregular access patterns cause frequent memory stalls due to low data locality. Dependent sequential memory access limits the available instruction level and memory level parallelism.

[☆] This work is supported by National NSF of China (61572194,61672233).

^{*} Corresponding author.

E-mail addresses: 52184501012@stu.ecnu.edu.cn (Q. Zhang), 51255902184@stu.ecnu.edu.cn (H. Song), 51255902111@stu.ecnu.edu.cn (K. Zhou), 52215902007@stu.ecnu.edu.cn (J. Wei), 52184501011@stu.ecnu.edu.cn (C. Xiao).

¹ <https://www.intel.com/content/www/us/en/docs/vtune-profiler/cookbook/2023-2/false-sharing.html>

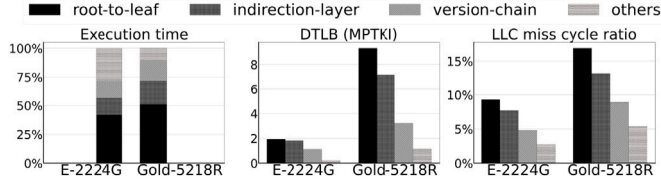


Fig. 1. Profiling results of pointer chasing codes vs. the rest of application codes when running YCSB-A workload (50% searches and 50% updates) on DBx1000 with different Intel Xeon processors, i.e., E-2224G(2017) and Gold 5218R(2019).

To quantify the performance impact of chasing pointers in real-world workloads, we run YCSB benchmarks [30] on DBx1000 [7], which is a popular open-source in-memory database system. We test different workloads for hardware profiling. The hardware profiling is carried out separately on different Intel Xeon processors, i.e., E-2224G and Gold 5218R. We use Intel® VTune™ profiling tool¹ to profile the database server process and analyze execution time and cache misses breakdowns. We run each evaluation for 5 minutes after it reaches a steady state. We profile the critical pointer chasing codes within the application separately from other parts of the application codes. Fig. 1 shows how pointer chasing compares to the rest of the application regarding execution time, data TLB cache misses per 10-kilo instructions (MPTKI), and the ratio of last-level cache (LLC) miss cycles to the total cycles.

On the processor E-2224G, we set up the database size with 1 GB. For each run, we spawn one thread and execute 100 million operations. On the processor Gold-5218R with a larger L3 cache, we set up the database size with 10 GB. For each run, we spawn ten threads and execute 100 million operations. Each bar in Fig. 1 is divided into four categories: root-to-leaf, indirection-layer, version-chain, and others. We make three significant observations. First, under both workloads, DBx1000 spends a significant fraction of its total execution time (75% and 82%, respectively) on pointer chasing, causing dependent cache misses. Second, we find that indirection-layer and version-chain increase the overall pointer chasing overhead, leading to a higher number of TLB misses (60% in E-2224G and 53% in Gold-5218R) and much higher cache miss rates (56% in E-2224G and 57% in Gold-5218R). Third, when running in a more recent processor, pointer chasing operations are also largely bound by main memory.

In this paper, we study the prefetching indexing scheme, which uses prefetching to limit the exposed miss latency. We implement the prefetching indexing scheme on DBx1000 in-memory database, and make it open-source.² In summary, this paper makes the following contributions:

- To the best of our knowledge, this is the first study to deeply explore how prefetching can accelerate search operations for in-memory MVCC database indexing.
- We combine a path prefetcher with a jump pointer prefetcher and propose the prefetching indexing scheme for hiding the memory latency induced by version searches.
- We achieve better performance on updates because our improved search speed more than offsets any increase in prefetching overhead.
- We begin to present a theoretical analysis of the memory access cost. Then, we perform experiments to compare our scheme with the state-of-the-art scheme, and prefetching indexing scheme outperforms the existing approach by 20% ~ 70% under YCSB workloads.

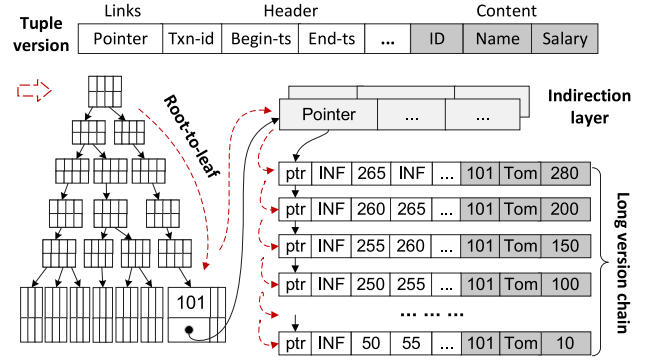


Fig. 2. Search in MVCC databases.

The remainder of this paper is organized as follows. Section 2 discusses the storage architecture of in-memory database systems and the challenges in the state-of-the-art indexing scheme. Section 3 presents the design of the prefetching indexing scheme. Section 4 presents the experimental evaluation results. Section 5 presents related work. Section 6 gives a detailed discussion of future research. This paper is concluded in Section 7.

2. Background and challenges

2.1. Storage architecture

For IMDBs, main memory is the primary storage location. Under MVCC, the system maintains multiple physical tuple versions of each tuple in the database. All MVCC database systems keep the database's versioning information separate from its indexes. The memory storage area for tables is split into multiple data blocks. A mapping table serves as an indirection layer that maps logical block IDs to physical data block addresses. Data blocks are always accessed via a mapping table lookup. Each data block consists of a set of slots. The system stores the tables' tuple versions in these slots, and tuple versions are unsorted within those data blocks.

Fig. 2 shows a simple user salary table. The table has three columns: ID (primary key), Name, and Salary. A tuple version includes a link pointer and header fields. A tuple version's valid time is defined by timestamps stored in the Begin-ts and End-ts fields in the header. The example table has a B⁺-tree index on ID. The leaf nodes of the B⁺-tree store pointers to tuples. If a tuple has multiple versions, the tuple versions are linked together using a link field in the tuples, and the index always points to the first version (HEAD) on the version chain.

The system uses an indirection layer (arrays) that maps each tuple to the head of its version chain. Then, the index entry points to the tuple's mapping entry instead of its physical location. It can reduce index maintenance costs significantly because only the mapping entry needs to change each time. However, for each tuple search, the indirection layer incurs additional pointer-chasing operations, making reads slower.

Suppose updating an existing tuple; the system first acquires an empty slot from the table for the new tuple version. It then copies the data of the current version to the new version. Finally, it applies the modifications to the tuple in the newly allocated version slot and installs the new version as the version chain HEAD. After that, the mapping entry of the indirection would be changed to point to the new version, while the index entry would not need to be updated.

² https://github.com/gitzhqian/DBx1000_prefetching.git

Table 1

A summary of the design decisions made for the state-of-the-art IMDBs' indexing schemes. (-: Irrelevant or Incompatible.).

	Tree-Index Structure	Multi-Version Storage Scheme	Version Referencing
Silo [1,2]	mass-tree	copy-on-write	directness
Hekaton [3]	bw-tree	append-tuples	indirection
Peloton [5]	bw-tree	append-tuples	indirection
Ermia [8]	mass-tree	append-tuples	indirection
Cicada [9]	mass-tree	append-tuples	best-effort inlining
InnoDB [10]	b ⁺ -tree	delta-tuples	inlining
Postgres [10]	b ⁺ -tree	append-tuples	indirection
DBx1000 [7,32,33]	b ⁺ -tree	append-tuples	indirection
Hyper [4,31]	–	delta-tuples	versioned positions
LeanStore [6]	b ⁺ -tree	–	–

2.2. Challenges for prefetching search

Previous work has surveyed index types of major commercial and academic in-memory DBMSs and found that B⁺-tree is the most popular index type. We will focus on B⁺-tree in this work. Table 1 summarizes the characteristics of the state-of-the-art IMDBs' indexing schemes [1–10,31–33]. (1) **Tree-index data structure**: presents default index structure type. Both Bw-tree and Mass-tree are B⁺-tree variations. (2) **Multi-version storage scheme**: specifies how the system stores these versions generated by the transaction's update operations. (3) **Version referencing**: used to map index entries to tuple versions in base tables. Considering the characteristics of modern storage technologies, append-tuples version storage and indirection are preferable, due to lower write amplification and higher parallelism. However, it tends to cause massive read amplification for mixed workloads.

Fig. 2 shows how IMDBs perform a search over versioned tuples. For example, transaction Txn-52 invokes a read operation on ID = 101, the system searches for a physical version (...50,55, ...,101,Tom,10) where Txn-52 is in between the range of the Begin-ts and End-ts fields. It must first follow the path ❶ (*root-to-leaf*) and visit six nodes in the path sequentially. Then, it must look up the path ❷ (*indirection layer*) and fetch the version chain HEAD according to the address. Thirdly, it must process a linear traversal of tuple versions following the path ❸ (*long version chain*).

On the one hand, generally, to find a visible tuple version of the transaction, a path❶-to-path❷-to-path❸ traversal is always needed; the nodes in the traversal path are visited sequentially. On the other hand, even for updating or deleting a tuple, it always needs to locate a tuple version chain HEAD. Moreover, most systems implement the Pointers as logical pointers, such as Tuple ID (a unique 64-bit tuple identifier comprised of block ID and offset); a hash table lookup is needed for each tuple version accessed. Such an indexing structure poses a significant challenge for prefetching because the version searches suffer from pointer-chasing problems [22].

Several earlier pointer-based prefetching approaches focused on prefetching for linked lists or trees, such as [21,25,26,34,35]. However, due to the vast diversity of traversal paths, it would not work well for IMDB indexing structures. Different from other simple data structures, on the one hand, the access sequence of IMDBs indexing searches is highly dependent on the input data, i.e., the search key, which almost cannot be predicted efficiently. On the other hand, the access patterns are more irregular. For example, the volume of tuple versions grows exponentially while the search is in the probing phase. It increases the difficulty of prefetching for IMDB indexing searches.

3. Design

We propose a practical approach to address two challenges in designing IMDBs prefetching indexing: identifying and predicting the access patterns by utilizing an online query monitor and accelerating pointer chasing using a path prefetcher combined with a jump pointer prefetcher. In this section, we first present a design overview of the prefetching indexing scheme. Then, we explain how to track the blocks that are accessed by each online query, how we use this information to generate prefetch candidates, and how we use jump pointer prefetching to accelerate linear version traversal. Lastly, we describe the database operations and cost analysis.

3.1. Overview

The design of the proposed prefetching indexing scheme is based on three observations.

First, recent hardware improvements can provide better performance to IMDBs with low overhead, such as very large DRAMs with Terabytes (TBs) capacity, high memory bandwidth, and larger last-level cache (LLC/L3 cache) [36–38]. However, the cache-to-memory latency gap can be a significant performance bottleneck for programs working on large data sets with poor data locality. An efficient approach that can reduce the number of cache misses and the impact of cache misses is desirable for improving search performance in in-memory MVCC systems.

Second, different from the linked list data structure, the access sequence of the tree structure is highly dependent on the search key, which increases the difficulty of prefetching for searches. Under those workloads in modern database applications, on the one hand, the latest tuples are likely to be frequently accessed (select/update) during a period [5]. On the other hand, searches of adjacent keys tend to follow similar traversal paths since keys are stored in sorted order [39]. Note that there are common blocks between their traversal paths. Then, the read counters of index blocks are necessary to make informed decisions on which blocks to be appended to the prefetch candidates.

Third, as prior studies demonstrated, many undesirable side-effects of MVCC result from a growing volume of tuple versions. Most database systems implement the version chain as a linked list using the tuples' *pointer* field. The *pointer* either points to the next tuple version in the chain or is *null* if there is none. To accelerate version chain traversal, we must first overcome the pointer-chasing problem.

Fig. 3 shows the logical overview of the prefetching indexing scheme. It consists of three components: online query monitor, access path prefetcher, and jump pointer prefetcher. We make the following design decisions.

Employ Path Prefetcher policy for tree search. Better performance can be achieved if each processing thread efficiently identifies and predicts the access pattern of tree index search operations. We employ an access path prefetcher policy for tree search. To make this policy work, we present three designs. For a search key 101, let a path P be a sequence of non-leaf nodes that are generated during tree traversal. Then, $P = \{p(l, i) | 1 < l < h - 1\}$, $p(l, i)$ represents the paths of accessing i th node at level l .

- **Cache Prefetching**: During the traversal, before starting a binary search on a node within the $p(l, i)$, we prefetch all of the cache lines that comprise the node.
- **Path Prefetching**: Before starting a search from the root node to the leaf node P , we prefetch the node blocks that comprise the sequence.
- **Monitor Prefetching**: We collect recent query statistics by leveraging a lightweight monitor that tracks the inner nodes that are accessed by each query. Then, those samples will be used to produce *Prefetch Table*, which records the frequent access paths for the future prefetches, e.g., $\{p(1, i - 2), p(1, i - 1), p(1, i), p(1, i + 1), p(1, i + 2)\}$.

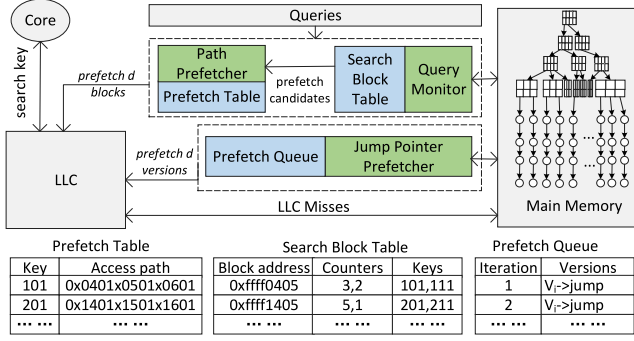


Fig. 3. The logical overview of prefetching indexing scheme.

Implement Leveled Query Monitoring according to the read counters. Each inner node has a read counter that is increased when the node receives a read request. An optimal query monitoring strategy should compute the total samples with the read counters. However, the overhead of computing the total samples offsets the benefit of the optimal strategy. Thus, we use a leveled query monitor to reduce the monitoring overhead. For the L_1 level of the tree, the monitor gathers statistics and dynamically generates the d (prefetching distance, in units of blocks to prefetch ahead) candidates according to the read counters. All inner nodes will be treated as prefetch candidates for the other levels.

Build Jump Pointer Prefetcher to accelerate in-chain version lookup. Since version traversal is commonly referred to as pointer chasing, we create a jump pointer queue (*Prefetch Queue*) and use it to accelerate linear version traversal. During an iteration of the v_{i+2} , the prefetcher directly prefetches v_i using two-ahead jump pointers. It can significantly improve the performance of pointer chasing by decreasing the number of expensive misses.

3.2. Path prefetcher

3.2.1. Cache prefetching

Cache prefetching is a common technique to overcome the speed gap between processor and memory and hide off-chip memory latency. There are two typical cache prefetching strategies: reducing the number of cache misses and reducing the impact of cache misses. Generally, a database workload exceeds the cache capacity, and the first strategy is to remove cache misses by improving data spatial locality. A well-known example is cache-conscious data layout, such as PAX [40], Data Morphing [41], and FSM [5] storage models. They need to modify the placement of data items so that references to multiple data items from the same cache line occur closely in time without incurring additional cache misses. Then, data reorganization should be controlled carefully to avoid false negatives. Moreover, cache misses can also result from accessing data for cold searches.

In this work, we focus on the second general strategy. The goal is to make cache misses less expensive by exploiting memory system parallelism. Modern processors allow multiple outstanding cache misses to be in flight simultaneously within the memory hierarchy and support prefetch instructions to load data into different levels of CPU caches [37]. We leverage the prefetch instructions to hide cache miss latencies.

Code Snippet 1(a) depicts the standard binary search in an inner node of the tree. For the B^+ -tree, each inner node contains 8B *key* fields and 8B *childptr* fields. Consistent with previous studies [42], we use the optimal node width $w_{optimal}$ for the cache prefetching. Our experiments show that the optimal node width is eight cache lines. Then, as depicted in Code Snippet 1(b), before starting a binary search, we issue prefetch instructions to read the data into the L3 cache (see Fig. 4).

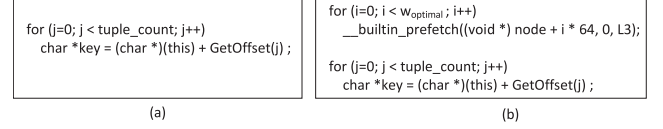


Fig. 4. Code Snippets 1: (a) standard binary search in a inner node. (b) binary search with cache prefetching.

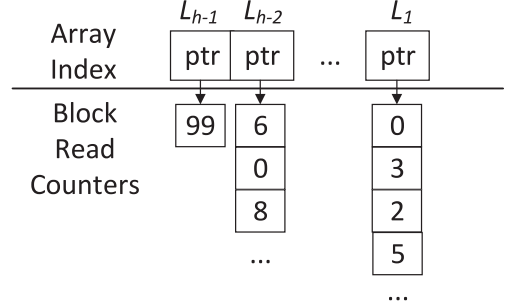


Fig. 5. Structure of Read Counters and Index.

3.2.2. Online query monitoring

We use a query monitor that tracks the blocks that are accessed by each query. The goal is to record reads to inner nodes (block); each block is associated with a read counter, and its value is increased by one when the block receives a read request. During the monitoring, search keys' access paths will be recorded in *Search Block Table*.

There are two alternative approaches to how the counters can be managed. First, the counter can be stored inside each block by utilizing the first 8 bytes of this block. The counter migrates to the newly allocated block as the key is redistributed. Take insertion as an example. When a leaf node splits, keys must be redistributed to addresses. If this split succeeds, two new child nodes and a new parent node are in the B^+ -tree structure. We can copy the current counter value to the newly allocated blocks.

One drawback of this approach is that when the counters of the child nodes are updated, cascading updates are propagated to higher-level nodes. This side effect makes it difficult to manage the counters efficiently. Another one is that we must perform a complete root-to-leaf traversal when gathering statistics from these blocks. However, this expensive operation should be avoided.

Second, the counters can be stored in a separate space and accessed by array indexes. During the query monitoring, the requested blocks of search keys are recorded in the *Search Block Table*. Then, a worker thread will build an index on the read counters based on these statistics. By looking up the index, the prefetcher can quickly determine which blocks to prefetch in the future.

Fig. 5 shows the structure of the index. The tree's inner nodes (block) are segmented by the tree levels. Each segment is organized as an array. The array index structure consists of *ptr* fields. *Ptr* points to the head node of the counter array, which stores read counters for the blocks of each level. The index nodes are sorted by block addresses.

The second strategy offers two advantages over the first one. On the one hand, accessing the counter array takes $O(1)$ time to get the counters by block number. On the other hand, the counters do not need to migrate as the blocks are allocated or freed. The counters are updated when the search reaches. In case of insertions, the worker thread will adjust the index structure dynamically.

3.2.3. Path prefetching

In in-memory MVCC databases, tree index data structures will never change with update operations of keys. The access paths of search keys will also never change. Given this observation, there is a naïve prefetching algorithm.

Algorithm 1: Monitor Prefetching Algorithm.

```

1 Function MonitorPrefetching(recent searches K,
  representative paths P)
2   #Stage 1: Clustering algorithm ;
3   foreach all keys k appearing in K do
4     foreach all representative keys rk within P do
5       if (rk is closest to k) then rk ← rk + w × (k − rk) ;
6     end
7   end
8   #Stage 2: Round-Robin algorithm ;
9   Generate prefetch instructions for L1 using r;
10 end

```

For each table index in the database, the system maintains statistics about the recent queries that have accessed it. For each query, the system extracts its accessed blocks, counter, and search keys. The path prefetcher periodically computes and identifies access paths for search keys using those query statistics. Over time, the *Prefetch Table* will hold all the keys' access paths.

When a search key normally hits the *Prefetch Table*, the prefetcher will prefetch all the blocks in the access paths. Only the blocks in the start part of the access path will be prefetched if partial hits.

3.2.4. Monitor prefetching

Holding all the keys' access paths online is infeasible in practice. We next describe how to compute workload-optimized access paths.

Computing the optimal access paths for a workload based on its query statistic requires exponential space and time in the number of blocks. Consider a B^+ -tree of 7 height and each inner node with 16 *key/childptr* pairs. The total number of inner nodes is 248,760, and the block size of L_1 level is 121MB (>L3 cache size).

Then, we adopt a more resource-efficient prefetching algorithm that converges toward the optimal access paths over time. Our approach leverages query statistics to compute the paths in two stages. As shown in Algorithm 1, it first employs a clustering algorithm to determine which blocks should be treated as candidates. It then uses a Round-Robin algorithm that uses the output of the clustering algorithm to generate workload-aware access path prefetch instructions.

For each search key, the clustering algorithm observes the accessed blocks and assigns them to the j th cluster, whose mean representative query r_k is most similar to k . The distance metric between two keys is defined as the number of common blocks of their traversal paths. Keys that access many common blocks, therefore, tend to belong to the same cluster. After assigning the search key to a cluster, it updates r_k to reflect the inclusion of the search key. The algorithm prioritizes each key path based on its total read counters, $w(k) = \sum_1^{h-1} (BlockCounter)$. The weight w determines the rate with which the algorithm forgets the older path samples.

In addition, the cluster's mean is a vector of block/counter pairs (comprised of the *Prefetch Table*) whose length is less than the $0.35 \times L3$ cache size. Over time, the means of the clusters drift towards the recent samples. Roughly speaking, we can estimate this L3-limited prefetching distance as $d = \frac{p \times B_{L3}}{B_{InnerNode}}$, where p is the percentage of L3 capacity occupied, B_{L3} is the number of bytes of the L3 cache, and $B_{InnerNode}$ is the number of bytes per inner node. With the clustering algorithm, the system maintains the frequent d candidates for future prefetches.

3.3. Jump pointer prefetcher

For in-memory database systems, the typical tuple version layout is shown in Fig. 6 (a). The tuple version chain is organized as a linked list. Most systems store the newest tuple version as the version chain's HEAD. The version consists of a header and links fields. The txn-id

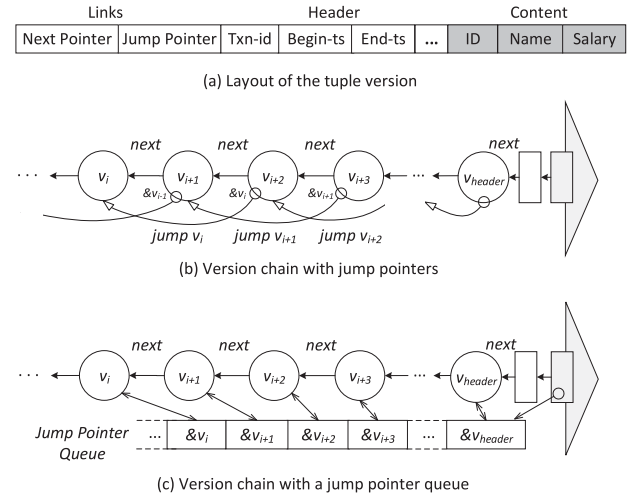


Fig. 6. Version Chain with jump pointers.

field serves as the version's write lock. The following two fields are the begin-ts and end-ts, representing the tuple version's lifetime. The next pointer field links the tuple version nodes together. The content field stores the tuple version data. To perform a read operation on a tuple A, the system searches for a visible version by comparing a transaction's read-timestamp with the begin-ts and end-ts fields.

As illustrated in Fig. 6 (b), assuming that the version v_i is visible to the currently active transaction t , when the leaf node is visited, we would like to launch a prefetch of v_i . We would generally follow the *next* pointer through the indirection-layer, v_{header} , ..., v_{i+3} , v_{i+2} , and v_{i+1} before the prefetch of v_i could be launched. However, fetching each node would incur much more full miss latency. There is a jump pointer prefetching algorithm to solve the pointer-chasing problem. This approach involves creating a new pointer field for each version, i.e., jump pointer, which points from a version to the one it should prefetch. For example, Fig. 6 (b) shows how v_{i+2} could directly prefetch v_i using a two-ahead jump pointer. The goal is to ensure that by the time the iteration is ready to visit a version, that version is already prefetched into the cache, thereby hiding much miss latency of v_i .

In this work, we will build upon the concept of jump pointers but customize them to the specific needs of version chains. First, we pull all version nodes' addresses into a separate contiguous space called the jump pointer queue, as illustrated in Fig. 6 (c). Second, to initiate prefetching, a new pointer field of the search key within the leaf node is used to locate the version chain's HEAD. Then, before starting the traversal of the version chain, we can issue prefetches for the first n versions. Using the jump pointer queue is more flexible than jump pointers stored directly in the tuple version bodies. We can adjust the prefetching distance by simply changing the offset used within the queue.

The critical issue in implementing a jump pointer queue is gracefully handling updates. The version chain's HEAD changes whenever a tuple is modified. One can avoid this problem through the queue's *cursor*, which always points to the physical address of the tuple's version header. With this setup, the jump pointer of a search key within the leaf node points to the *cursor* of its jump pointer queue instead of the version header's physical address. In addition, in our implementation, the queue is broken into a chunked linked list, which allows us to limit the impact of a version insertion to its corresponding chunk. We configure the chunk size as 8 (one cache line) in our experiments.

3.4. Operations

Update: Most transactions update the latest version of a tuple. Since an index search is first performed to locate the position for an update

operation, all of the blocks on the access path from the root-to-leaf to the version chain's HEAD are already in the cache before the real update processing. When updating an existing tuple, the transaction first creates a new version if no active transaction holds the current version's write lock and its transaction ID is larger than the current version's last reader field value. If these conditions are satisfied, the system acquires an empty slot from the table for the new version. It then copies the content of the current version to the new version. Finally, it applies the modifications to the tuple in the newly allocated version slot. When the transaction commits, the system sets the new version's Begin-ts and End-ts fields and maintains the Next Pointer and Jump Pointer fields.

Search: To perform a read operation on a tuple, the searching first traverses the root-to-leaf paths in B^+ -tree; once searching reaches a particular leaf node containing the search key, it executes a linear version traversal through the *next* link from there. Then, it searches for a visible version by comparing the current transaction's read-timestamp with the versions' begin-ts and end-ts fields. Our search algorithm is a straightforward extension of the standard IMDB tree-index search algorithm with two-fold changes. First, before starting a binary search of the tree, we prefetch the node blocks in the traversal path. Second, when executing a linear traversal of tuple versions, we perform prefetching to fetch multiple version blocks ahead. This approach can help accelerate the search, scan, and update operations of an IMDB indexing structure. Moreover, under modern database workloads, these operations occur frequently.

Range Scan: The basic index-only range scan algorithm consists of a loop that visits a leaf node on each iteration by rechecking the last value in the iteration. The value represents the largest key of the previous leaf node and will allow the traversal to find the next leaf node position in the scan. By leveraging the query statistics, we add prefetches both at the startup and inside the loop.

3.5. Cost analysis

In this section, we provide qualitative analysis to show that our prefetching indexing scheme achieves a better version search performance. At the core, version searching resembles a combination of tree-based and list-based irregular access patterns, firstly traversing the leaf-to-root paths in binary search trees; once version searching reaches the version chain HEAD, it performs regular linear traversals from there. Below, we analyze and model these steps in detail. All notation is summarized in Table 2.

Tree Index Traversal. For a B^+ -tree index with fanout b the index traversal cost, from the root to the first corresponding leaf, is a function of the number of key/tuplePointer pairs N and is given by:

$$TIT = (1 + \lceil \log_{mw_i} N \rceil) \cdot (T_{ilb} + (\frac{w_i}{2}) \cdot T_1) \quad (1)$$

$$TIT_p = (1 + \lceil \log_{mw_i} N \rceil) \cdot (T_1 + (\frac{w_i}{2} - 1) \cdot T_{next}) \quad (2)$$

Eq. (1) assumes that traversing from a node to its child results in a random memory access, which will be incurred based on the height of the tree, and that, on average, for each non-leaf node and leaf node, half of the keys will be read sequentially. In the prefetching indexing design, before starting binary searching, we perform path prefetching, and then the node blocks can be cached in the TLB. Assume that each node has multiple cache lines, the cost of the first cache line is T_1 , and the additional latency of fetching the following cache line in parallel is T_{next} . As shown in Eq. (2), the costs of accessing a node depend on the cache miss that can be served in parallel in the memory system.

Version Chain Traversal. After reaching the leaf node, the cost of accessing the indirection arrays is a cache miss and a TLB miss, and we include them in this model. However, this cost can be reduced in the prefetching indexing design due to the jump pointer prefetching. Since the version nodes can be in arbitrary memory locations, each

Table 2
Terminology.

Variable	Definition
N	data size, key/tuplePointer pairs in a tree
b	tree fanout, $b = m \times w_i$
w_i	number of cache lines in a tree node
m	number of key/pointer pairs in one cache line
n	number of versions of a tuple
w_v	number of cache lines in a version node
T_1	RAM(L1-to-Memory) latency of a cache miss
T_{next}	latency of an additional pipelined cache miss
T_{tlb}	latency of a data TLB miss

time we need to visit a tuple version, a cache miss and a TLB miss are incurred. Thus, given that n is the number of versions of a tuple, and i -th is the required version, the version chain traversal cost for each version searching is:

$$VCT = T_{tlb} + T_1 + n_i \cdot (T_{tlb} + (\frac{w_v}{2}) \cdot T_1) \quad (3)$$

$$VCT_p = n_i \cdot (T_1 + (\frac{w_v}{2} - 1) \cdot T_{next}) \quad (4)$$

Putting together Eqs. (1) and (3), the cost of a version searching using traditional indexing is:

$$IndexingCost = TIT + VCT \quad (5)$$

Putting together Eqs. (2) and (4), the cost of a version searching using prefetching indexing becomes:

$$PrefetchingIndexingCost = TIT_p + VCT_p \quad (6)$$

Here, we compare the traditional indexing and the prefetching indexing for version searching using the above equations. Based on our experimental setup, the parameters are as follows: $T_{ilb} = 18$ cycles, $T_1 = 200$ cycles, $T_{next} = 8$ cycles [43]. For $N = 10$ million tuples database size, $m = \frac{64 \text{ bytes}}{16 \text{ bytes}} = 4$, we can compute the optimal w_i value by minimizing the tree index traversal cost, that the optimal w_i would be 4, 6, 8, 10, 12. We consider a version chain length more than 10 and a tuple version size more than 200 bytes, then $w_v = \frac{200 \text{ bytes}}{64 \text{ bytes}} \approx 3$. According to Eq. (5), in the traditional indexing design, the overall cost of a version search equals $(1 + 6) \times (18 + \frac{4}{2} \times 200) + 18 + 200 + 10 \times (18 + \frac{2}{2} \times 200) = 6324$ cycles. According to Eq. (6), in the prefetching indexing design, the overall cost of a version search equals $(1 + 6) \times (200 + (\frac{4}{2} - 1) \times 8) + 10 \times (200 + (\frac{2}{2} - 1) \times 8) = 3440$ cycles. Therefore, comparing our scheme with the state-of-the-art scheme, we see that the prefetching indexing scheme is at least $\frac{6324}{3440} \approx 1.84\times$ better than the traditional design.

Space Overhead. All frequent access paths' storage space overhead is (Number Of Blocks In Those Paths) $\times$ 8B. In a 16 key fields leaf node, the space overhead of all the metadata (jump pointers) is $16 \times 8B$ per 1536B sized block, i.e., $\frac{128B}{1536B} = 8\%$. The space overhead of the jump pointer queue is $\frac{8B \times n}{200B \times n} = 0.04\%$. However, the space overhead is relatively small since the garbage collector reclaims all the jump pointers and versions together.

4. Evaluation

In this section, we compare the prefetching indexing scheme with the state-of-the-art indexing scheme. We have implemented the prefetching indexing scheme in *DBx1000*,² a popular in-memory DBMS used to evaluate many IMDB technologies [7]. All transactions are executed with the default snapshot isolation level. We evaluate the prefetching indexing scheme on two CPUs: an Intel Coffee Lake CPU [44] and an Intel Cascade Lake CPU [45] with 28MB L3, as shown in Table 3. Both servers run 64-bit Ubuntu 20.04 with Linux kernel v5.15.0. We use

Table 3

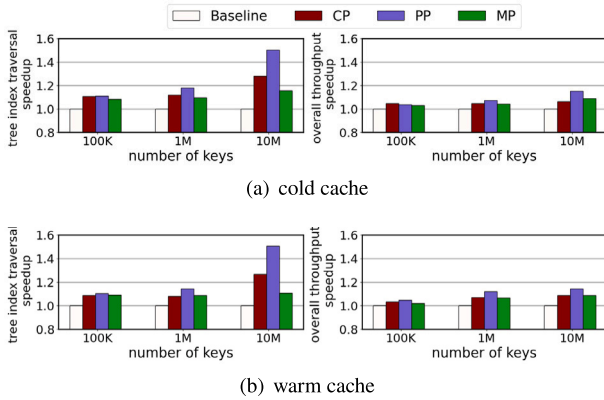
Parameters of the evaluated machines.

Processor1	Workstation, Intel Xeon E-2224G, 3.5 GHz
Cores	Coffee Lake, 2017, 4 cores
L1 Cache	Private 32 KB, 8-way, 4 cycles
L2 Cache	Private 256 KB, 4-way, 12 cycles
L3 Cache	Shared 8 MB, 16-way, 42 cycles
Memory	32 GB, DDR4-2666, 39.74 GB/s, 2 chanel, RAM latency: 220 cycles
Processor2	Server, Intel Xeon Gold 5218R, 2.1 GHz
Cores	Cascade Lake, 2019, 20 cores
L1 Cache	Private 32 KB, 8-way, 4 cycles
L2 Cache	Private 1024 KB, 16-way, 14 cycles
L3 Cache	Shared 28 MB, 11-way, 50 cycles
Memory	96 GB, DDR4-2667, 119.21 GB/s, 6 channels, RAM latency: 200 cycles [46]

Table 4

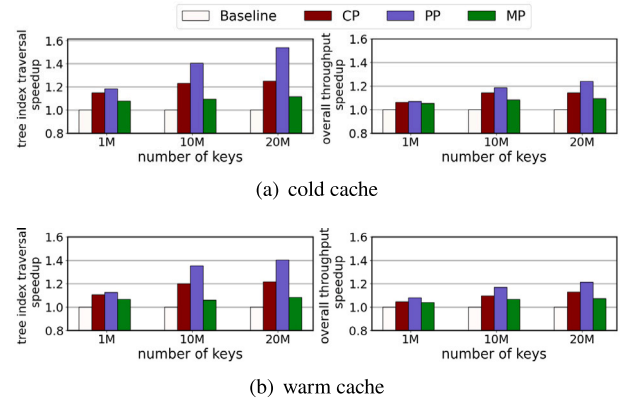
The number of keys, levels and nodes in trees (non-leaf node size = 512B, leaf node size = 1536B).

Number of Keys	Height	Number of Non-leaf Nodes	Number of Leaf Nodes
0.1 M	5	1734	13 029
1 M	6	16 258	131 005
10 M	7	248 760	1 310 653
20 M	7	438 752	2 621 290

**Fig. 7.** Search performance with various database size on Processor1. (We load a database with 100 K/1M/10M tuples, then perform key searches in a single thread under the different prefetching strategies, i.e., Baseline:no prefetching, CP: cache prefetching, PP: path prefetching, MP: monitor prefetching.)

version 11.1.0 of the gcc compiler with the “-O3” optimization flag.³ We use the Intel Xeon Gold 5218R as our main evaluation platform.

We used a set of Yahoo! Cloud Serving Benchmark (YCSB) benchmarks [30]. It contains a single table of tuples with an 8B primary key and ten columns of random string data, each 10B in size. Thus, each tuple version is more than 100B size. YCSB uses a Zipfian distribution to select keys for operations. The workload consists of three operation types: (1) a search operation that retrieves a single tuple based on its primary key, (2) an update operation that modifies a single tuple based on its primary key, and (3) a scan operation that starts from a random starting offset and then returns at most 30 matching tuples. The YCSB workload reports each operation type’s latency and number of operations. If not mentioned otherwise, we use a database with 10 million tuples (~1 GB).

**Fig. 8.** Search performance with various database size on Processor2. (We load a database with 1M/10M/20M tuples, then perform key searches in a single thread under the different prefetching strategies, i.e., Baseline:no prefetching, CP: cache prefetching, PP: path prefetching, MP: monitor prefetching.)

4.1. Search performance

In the following, we vary the database size, the hardware configuration, the prefetching distance, the node size, the number of threads, and the key access distribution to show the search performance of the prefetching indexing scheme under a variety of settings.

Varying the database size. In this set of experiments, we bulk load a database to have 0.1 million, 1 million, and 10 million tuples. Then, we perform 10 million search operations uniformly with a single thread. Table 4 depicts the number of levels and nodes in the tree structure.

Fig. 7 shows the performance of different prefetching strategies when running uniform searches on the three databases. The results are normalized to the baseline with no prefetching. For cold caches, the speedup of the prefetching indexing over the baseline is between a factor of 1.08 and 1.50. Likewise, the warm cache speedup is 1.07 to 1.48. We see that indexing with prefetching support performs better than indexing with no prefetching. We also find that the larger the database size, the higher the performance gains from prefetching indexing. Another observation worth mentioning is that path prefetching performs better than other designs. We conclude that the performance gains stem mainly from the accurate prefetch candidates for future keys.

Varying the hardware configuration. In the following, we evaluate the workload performance using the processor2 hardware. Fig. 8 shows the performance of 100 million uniform searches after bulk loading 1 million, 10 million, and 20 million keys into the database. On the server-grade CPU with a larger cache capacity and higher memory bandwidth, all three prefetching designs significantly improve performance. For cold caches, the speedup of the path prefetching over the baseline is between a factor of 1.18 and 1.53. In particular, the overall throughput speedups over the baseline are 1.14 for CP, 1.23 for PP, and 1.10 for MP on average. With the larger L3 cache, we can record much more prefetch candidates.

From the results, we can see that the higher the tree structure, the larger the search cost; when trees are of the same height, the more non-leaf nodes yield better performance. For CP and PP, larger L3 allows the prefetchers to issue more prefetch instructions, resulting in notable benefits. However, for MP, the more non-leaf nodes may incur more prefetching overhead for generating optimal prefetch candidates.

Varying the prefetching distance. Fig. 9 shows the tree traversal and overall throughput of 100 million uniform searches after bulk loading 10 million tuples into the database. For this experiment, we increase the L3 cache space occupied by prefetching from 10% to 55% to vary the number of prefetch candidates. For the different number of

³ <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

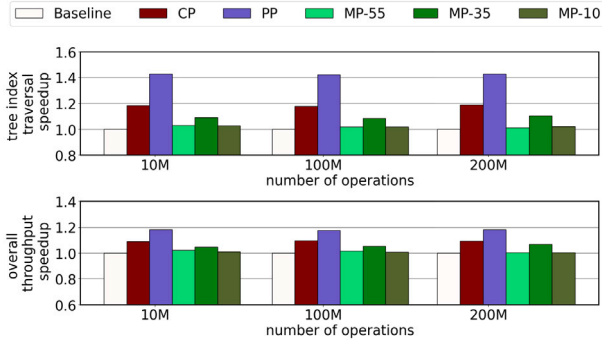


Fig. 9. Search performance with various prefetching distance (MP-55: prefetch candidates occupy 55% of the L3 cache capacity, MP-35: prefetch candidates occupy 35% of the L3 cache capacity, MP-10: prefetch candidates occupy 10% of the L3 cache capacity).

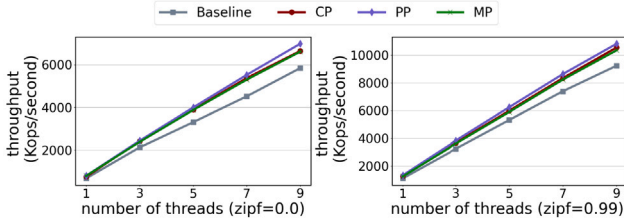


Fig. 10. Search performance with various threads.

operations, we see that the speedup of the MP-10 over the baseline is between a factor of 1.01 and 1.02. The speedup of the MP-55 over the baseline is between a factor of 1.01 and 1.03. The MP-35 achieves better performance (1.08–1.10 speedups over baseline). This is because the smaller the number of prefetch candidates, the less performance improvement can be achieved. However, prefetching a larger number of candidates would pollute the L3 cache, decreasing the overall throughput performance.

Varying the number of threads.

In this set of experiments, we further evaluate the impact of cache size on the indexing scheme using read-intensive workloads. Fig. 10 reports the search throughput while increasing the number of threads from 1 to 10. We bulk load a database with 10 million tuples. Then, every working thread performs 10 million search operations. We see that all indexing structures show almost linear performance scaling as the number of threads from 1 to 10. Overall, since most of the key accesses are directed to a few data blocks with the Zipfian $\theta=0.99$ skew, we observed that on the significant skew workload, the overall throughput is higher due to hot caches.

For no skew (Zipfian $\theta=0.0$) key accesses, we see that prefetching structures improve throughput by up to 20% under 10 threads. Under the significantly skewed workload, the benefits of prefetching slightly shrink. Overall, prefetching indexing is robust to skew because it deeply integrates prefetching into the version search processing.

Varying the node size. Fig. 11 reports search performance while varying inner node size from 512B to 2048B. Table 5 shows the number of accessed blocks in each tree level with the different node sizes. From the figure, we see that MP achieves a relatively stable throughput performance in all cases of different inner node sizes.

Comparing the different inner node sizes, we see that 512B is slightly better than the other sizes when combined with our prefetching scheme. The larger the inner node, the less the search performance improvement. It is because the larger the inner node, the smaller the number of levels of the tree, and the higher the search cost within the node, resulting in more minor benefits of prefetching optimizations. Overall, 512B is a good choice of inner node size for the prefetching

Table 5

The number of accessed blocks in each tree level when the inner node size changes (database size = 20 million tuples, tree height = 7/6/5).

Level	512B	1024B	2048B
$h-3$	69(0.03 MB)	113(0.1 MB)	985(2 MB)
$h-4$	1003(0.4 MB)	2910(3 MB)	49679(97 MB)
$h-5$	14107(7 MB)	102671(102 MB)	N/A
$h-6$	204196 (99 MB)	N/A	N/A

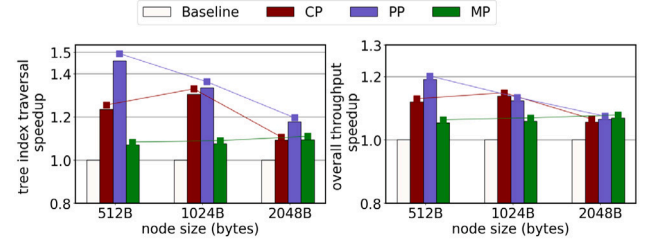


Fig. 11. Search performance with various inner node size.

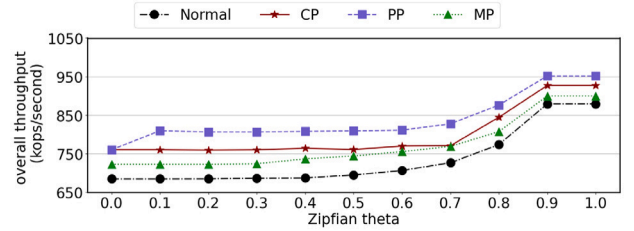


Fig. 12. Search performance with various zipfian coefficient.

indexing scheme. CP and PP are more sensitive to inner node size because the prefetch instructions cost expands during each iteration, causing longer latency for the search key.

Zipfian key access distribution. Fig. 12 presents the overall throughput (operations per second) of prefetching indexing on workloads with Zipfian distribution. The Zipfian distribution has been verified in characterizing the content popularity, and the θ is the Zipf parameter representing the skewness in content popularity. For lowly skewed access patterns, the speedup of prefetching indexing over the baseline is 1.20, 1.36, and 1.08 on average, respectively. For highly skewed access patterns, since most of the key access is direct to a few inner nodes with the Zipfian distribution, all benefits of prefetching optimizations decrease.

4.2. Range scan performance

We next examine the performance of prefetching indexing on range scan operations. We configure the scan predicate's key range to start the scan operation from a uniformly random starting offset. We vary the number of keys to scan per request (i.e., the size of the range).

From Fig. 13, we observe that when scanning only 10 or 20 keys, CP and PP prefetching designs are faster than MP. However, when scanning far fewer than 30 keys, the cost of our prefetching scheme becomes noticeable. When the request returns at most 30 tuples, there are at least three different leaf nodes on the traversal path. Generally speaking, these leaf nodes have common inner nodes. It may introduce duplicate prefetching instructions, causing wasted space and CPU cycles. On the other hand, when scanning more leaf nodes, the key issue is reducing the search cost within the leaf nodes instead of the access paths. For example, by just only prefetching ahead these leaf nodes (i.e., LP: leaf node prefetching), we see a speedup of 1.14 over the baseline in this case.

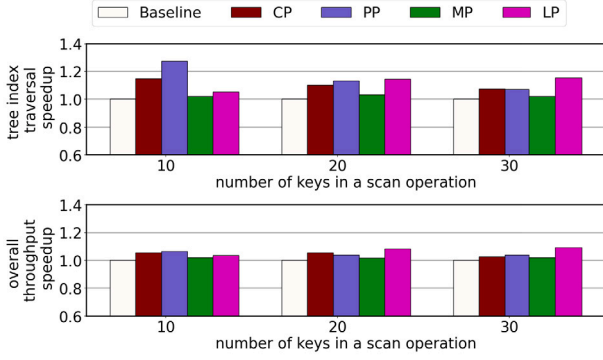


Fig. 13. Search performance with various number of keys in a scan operation (database size=10 million tuples, 1 million number of transactions, 10 operations per transaction).

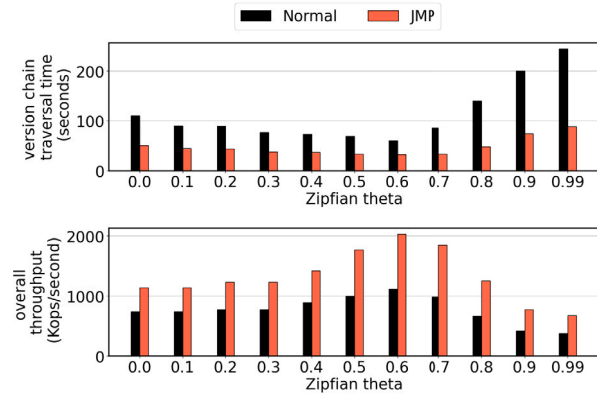


Fig. 14. Search performance with various version chain length. (database size=1 million tuples, 10 working threads, each thread performing 0.01 million transactions, 100 operations per transaction (50% searches/50% updates)).

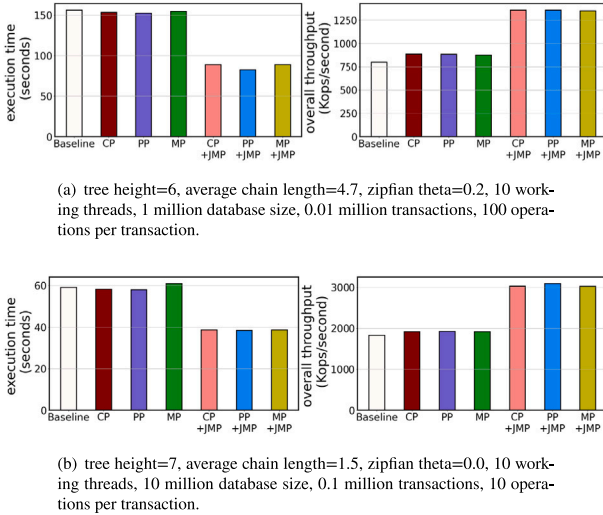


Fig. 15. Search performance with different prefetching designs (Baseline:no prefetching, CP: cache prefetching, PP: path prefetching, MP: monitor prefetching, CP+JMP: cache prefetching combined with jump pointer prefetching, PP+JMP: path prefetching combined with jump pointer prefetching, MP+JMP: monitor prefetching combined with jump pointer prefetching).

4.3. Update performance

In addition to improving search and scan performance, one of our goals is to achieve good performance on update operations. In

Table 6

The Zipfian theta and the average chain length accessed ($\text{length} = \frac{\text{TotalChainLengths}}{\text{NumberOfSearchKeys}}$).

theta	0.3	0.4	0.5	0.6	0.7	0.8	0.9	0.99
length	4	4	4	4	5	5	5	6

this experiment, we use the update-intensive workload (YCSB-A: 50% searches, 50% updates). We vary the Zipfian distribution of the keys searched to test the version chain traversal performance. The Zipfian coefficient and the average chain length accessed are shown in Table 6.

Generally, increasing the length of the chains means that transactions take longer to execute, thereby increasing the likelihood that a transaction will conflict with another one. We set all transactions executing with the default snapshot isolation level. Read uses a transaction's identifier to search for a visible version of a tuple by traversing the version chain.

As shown in Fig. 14, under low contention (theta=0.2), the jump pointer prefetching can achieve 1.75× better performance over the normal scheme because it reduces the impact of significant misses. This phenomenon is also evident with tests under the highest contention level (theta=0.99), where the average chain length is 6, and jump pointer prefetching achieves 1.58× better performance.

4.4. Prefetching performance

Finally, our last set of experiments presents an overall prefetching performance study. A central claim of this paper is that the demonstrated speedups for prefetching indexing are obtained by effectively limiting the exposed miss latency of previous approaches. In this experiment, we confirm that claim and show the contribution of both aspects of our prefetching indexing design to overall performance.

In the first experiment, we bulk load 1 million tuples and then run 10 work threads, each one performing 0.01 million transactions. There are 100 operations per transaction mixture comprising 50% searches and 50% updates. These operations access tuples following the Zipfian theta=0.2 distribution. As shown in Fig. 15(a), for the overall throughput performance, we can see that the prefetching indexing scheme (path prefetcher combined with jump pointer prefetcher) can achieve 1.76× speedups compared with the baseline (without any prefetching optimization).

In the second experiment, we bulk load 10 million tuples and then run 10 work threads, each one performing 0.1 million transactions. There are 10 operations per transaction mixture comprising 50% searches and 50% updates. These operations access tuples following the Zipfian theta=0.0 distribution. As shown in Fig. 15(b), for the overall throughput performance, we can see that the prefetching indexing scheme (path prefetcher combined with jump pointer prefetcher) can achieve 1.65× speedups compared with the baseline (without any prefetching optimization).

5. Related work

IMDBs Indexing Scheme. Several works from industry [3,19,47, 48] and academia [2,4–7] propose designs for IMDBs indexing schemes. All of these proposals suffer from the pointer chasing we highlight in Section 2, and none can fully exploit memory system parallelism in today's processors, which results in a high cost of memory access. Our prefetching indexing scheme exploits prefetching techniques, i.e., path prefetcher and jump pointer prefetcher, for enhancing CPU cache performance for in-memory MVCC index searches.

Prefetching for Pointer-Based Data Structure. Many data analytic and machine learning kernels in data center workloads operate on massive, memory-resident, pointer-based data structures (i.e., lists, trees, and graphs). Various mechanisms have been proposed to prefetch data in pointer-based data structures to hide the cache miss latencies.

These approaches are hardware-based [27], software-based [24,26,28,49] or combined software-hardware [23,25,29] mechanisms. This paper focuses on only the software-based in-memory MVCC indexing accelerating strategy. The software approach offers excellent flexibility that can be applied to any hardware platform. One strength of our scheme is its generic applicability in that one can apply prefetching indexing to any MVCC indexing storage.

Version Search Structure. While multi-version is the most common design in modern database systems, it has inherent problems in read-update workloads. If there are many updates in the presence of long-running queries, the length of version chains grows quickly. IMDB engines have faced such a challenge long before and have overcome it through finer-grained garbage collection [50]. Recently, work [51] proposes vWEAVER, which incrementally constructs a logical version search structure over the physical version store and allows a scan to traverse new version search structures for fast lookup. In this work, we design a cooperative prefetching model, i.e., path prefetcher and jump pointer prefetcher, to accelerate pointer-chasing operations over multi-versioned data.

6. Discussion and future work

Memory Wall is still a bottleneck in enhancing the performance of computer systems now. How to reduce *Memory Access Latency* is a problem that we have to deal with. Assuming that minimizing the memory access latency problem is *NP-complete*, pursuing optimal query performance is a long journey for in-memory database systems. While integrating path prefetcher and jump pointer prefetcher into the popular in-memory database storage engine, we faced difficult decisions. This section summarizes them as lessons one should carefully consider when adopting our designs in others' codebases.

- **Worst-case space complexity.** Generally, we expect better search performance, comparable or better update performance, and lower space overhead. The prefetcher has two inevitable factors fueling memory space expansion: a series of access block addresses and jump pointers. We quantify the extra space overhead in the DBx1000 storage engine (detailed in Section 3.5). In fact, besides the more extensive data set, frequent update operations incur much more significant overhead. In the worst case, the space overhead could increase linearly. However, this approach is attractive concerning space overhead because of the low overall space overhead. The overhead of jump pointers should be insignificant because the engine's garbage collection (GC) component will reclaim expired versions periodically. It ensures that the version chain's length is reasonable at any point.
- **To prefetch, or not to prefetch.** This paper focuses on leveraging the software prefetching technique to accelerate version searches. In addition to prefetch instruction overhead, there are further threefold limits to the benefit of our prefetch approaches. (1) A small table, for instance, does not have much advantage from prefetching because most of the data is in the caches of the CPU. Thus, we are unable to hide any latency. With increasingly growing table sizes, we hit the point where even the L3 cache (the last level cache in our experiments) is exhausted. We achieve the highest performance gain. (2) Since the L3 cache resource is limited, our current approach seems inefficient when one search touches a very long access path that exceeds the threshold, e.g., L3 cache size. Pipelined prefetching, paralleled prefetching, and grouping prefetching are worth the effort. (3) When to issue the prefetch instructions significantly impacts the prefetching performance. It is worth revisiting the query optimization with new version search methods to achieve a significant synergy effect.

- **Hardware/software prefetcher.** Some database operations (e.g., range queries, loop iteration, and hash joins) are computationally intensive, and hardware-based or hardware/software-cooperative prefetchers are required for good performance. However, depending on application types, workload differs amongst database functionalities. Thus, resources can be wasted or improperly allocated if the database system does not have fine-grained elasticity. Moreover, these prefetchers need help to detect more complex data structures, and they cannot perform software-specific optimizations (e.g., prefetching for future iterations of nearest neighbors instead of direct successors).
- **Learning-based prefetching indexing scheme.** While using an in-memory histogram to track the workload patterns is significantly efficient in our design, it seems unintelligent. Assuming that a prefetcher *learns* different access paths and predicts the workload patterns, it could *predictively adapt* its prefetch candidates. First, it predicts that an area of data will be accessed in the future; it treats that area as prefetch candidates soon before access occurs. Second, it predicts resource contention and accesses' characteristics to enable automatic change in the prefetching strategies (e.g., pipelined or paralleled).
- **Searching unstructured data.** According to recent research, unstructured data, such as text, images, and video, took up about 80% of the 40,000 exabytes of new data generated in 2020, and their percentage keeps rising due to the increasing amount of human-generated rich media [52,53]. With the development of learning-based embedding models, embedding vectors are widely used for analyzing and searching unstructured data. The vectors are also continuously updated when new user/content comes, user behavior changes or the embedding model is updated. To provide fast online data ingestion, the vector database systems would append updates in parallel (e.g., multi-version concurrency control). Without an efficient version search method, query processing conducts searches using brute-force scans so that updates can be searched quickly. There are three challenges to integrating our designs (i.e., target two-dimensional database tables) into these systems: high-dimensional vector space, complex search semantics, and vector search costs.

7. Conclusion

We present a novelty *version search structure* that we call a prefetching indexing scheme. This scheme targets any in-memory MVCC system; its append-based update behavior benefits the lower write amplification and the higher parallelism but causes massive read amplification for mixed workloads. The prefetching indexing scheme combines a path prefetcher with a jump pointer prefetcher to accelerate the searches for the most recently updated tuples. In other words, this scheme explores how prefetching could hide the data cache miss latencies of searching visible versions of the given transactions. We evaluate the scheme on YCSB using synthetic datasets and workloads. On YCSB, it outperforms the traditional indexing scheme by $1.1\times - 1.5\times$ under read-only workloads and by up to $1.7\times$ under mixed workloads. We have open-sourced prefetching indexing scheme².

CRedit authorship contribution statement

Qian Zhang: Conceptualization, Funding acquisition, Methodology, Project administration, Software, Supervision, Visualization, Writing – original draft, Writing – review & editing. **Haoyun Song:** Formal analysis, Investigation, Validation. **Kaiyan Zhou:** Resources, Software. **Jianhao Wei:** Data curation, Software. **Chunqiao Xiao:** Project administration, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

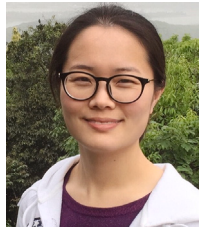
Data availability

Data will be made available on request.

References

- [1] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, Samuel Madden, Speedy transactions in multicore in-memory databases, in: Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles, 2013, pp. 18–32.
- [2] Wenting Zheng, Stephen Tu, Eddie Kohler, Barbara Liskov, Fast databases with fast durability and recovery through multicore parallelism, in: 11th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 14), 2014, pp. 465–477.
- [3] Cristian Diaconu, Craig Freedman, Erik Ismert, Per-Ake Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, Mike Zwilling, Hekaton: SQL server's memory-optimized OLTP engine, in: Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data, 2013, pp. 1243–1254.
- [4] Alfons Kemper, Thomas Neumann, HyPer: A hybrid OLTP&OLAP main memory database system based on virtual memory snapshots, in: 2011 IEEE 27th International Conference on Data Engineering, IEEE, 2011, pp. 195–206.
- [5] Joy Arulraj, Andrew Pavlo, Prashanth Menon, Bridging the archipelago between row-stores and column-stores for hybrid workloads, in: Proceedings of the 2016 International Conference on Management of Data, 2016, pp. 583–598.
- [6] Viktor Leis, Michael Haubenschild, Alfons Kemper, Thomas Neumann, LeanStore: In-memory data management beyond main memory, in: 2018 IEEE 34th International Conference on Data Engineering, ICDE, IEEE, 2018, pp. 185–196.
- [7] Xiangyao Yu, George Bezerra, Andrew Pavlo, Srinivas Devadas, Michael Stonebraker, Staring into the Abyss: An Evaluation of Concurrency Control with One Thousand Cores, Association for Computing Machinery (ACM), 2014.
- [8] Kangyeon Kim, Tianzheng Wang, Ryan Johnson, Ippokratis Pandis, Ermi: Fast memory-optimized database system for heterogeneous workloads, in: Proceedings of the 2016 International Conference on Management of Data, 2016, pp. 1675–1687.
- [9] Hyeontaek Lim, Michael Kaminsky, David G. Andersen, Cicada: Dependably fast multi-core in-memory transactions, in: Proceedings of the 2017 ACM International Conference on Management of Data, 2017, pp. 21–35.
- [10] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, Andrew Pavlo, An empirical evaluation of in-memory multi-version concurrency control, Proc. VLDB Endow. 10 (7) (2017) 781–792.
- [11] Cisco Global Cloud Index, Forecast and methodology, 2016–2021 white paper, 2018, Updated: February 1.
- [12] Shaosheng Cao, XinXing Yang, Cen Chen, Jun Zhou, Xiaolong Li, Yuan Qi, Titant: Online real-time transaction fraud detection in ant financial, 2019, arXiv preprint arXiv:1906.07407.
- [13] Xiafei Qiu, Wubin Cen, Zhengping Qian, You Peng, Ying Zhang, Xuemin Lin, Jingren Zhou, Real-time constrained cycle detection in large dynamic graphs, Proc. VLDB Endow. 11 (12) (2018) 1876–1888.
- [14] Van-Dai Ta, Chuan-Ming Liu, Goodwill Wandile Nkabinde, Big data stream computing in healthcare real-time analytics, in: 2016 IEEE International Conference on Cloud Computing and Big Data Analysis, ICCCBDA, IEEE, 2016, pp. 37–42.
- [15] Ronald Barber, Christian Garcia-Arellano, Ronen Grosman, Guy Lohman, C. Mohan, Rene Muller, Hamid Pirahesh, Vijayshankar Raman, Richard Sidle, Adam Storm, et al., Wiser: A highly available HTAP dbms for iot applications, in: 2019 IEEE International Conference on Big Data, Big Data, IEEE, 2019, pp. 268–277.
- [16] Anuruddha Thennakoon, Chee Bhagyan, Sasitha Premadasa, Shalitha Mithiranga, Nuwan Kuruwitaarachchi, Real-time credit card fraud detection using machine learning, in: 2019 9th International Conference on Cloud Computing, Data Science & Engineering, Confluence, IEEE, 2019, pp. 488–493.
- [17] Per-Åke Larson, Adrian Birka, Eric N. Hanson, Weiyou Huang, Michal Nowakiewicz, Vassilis Papadimos, Real-time analytical processing with SQL server, Proc. VLDB Endow. 8 (12) (2015) 1740–1751.
- [18] Ms P.R.S.M. Lakshmi, Ms K. SanthiSri, Mr N. Veeranjanyulu, A study on real-time business intelligence and big data, in: Information Engineering, vol. 4, Science and Engineering Publishing Company, 2015, pp. 1–6.
- [19] Franz Färber, Norman May, Wolfgang Lehner, Philipp Große, Ingo Müller, Hannes Rauhe, Jonathan Dees, The SAP HANA database—an architecture overview, IEEE Data Eng. Bull. 35 (1) (2012) 28–33.
- [20] Tirthankar Lahiri, Shasank Chavan, Maria Colgan, Dinesh Das, Amit Ganesh, Mike Gleeson, Sanket Hase, Allison Holloway, Jesse Kamp, Teck-Hua Lee, et al., Oracle database in-memory: A dual format in-memory database, in: 2015 IEEE 31st International Conference on Data Engineering, IEEE, 2015, pp. 1253–1258.
- [21] Chi-Keung Luk, Todd C. Mowry, Compiler-based prefetching for recursive data structures, in: Proceedings of the Seventh International Conference on Architectural Support for Programming Languages and Operating Systems, 1996, pp. 222–233.
- [22] Chi-Keung Luk, Todd C. Mowry, Automatic compiler-inserted prefetching for pointer-based applications, IEEE Trans. Comput. 48 (2) (1999) 134–141.
- [23] Gabriel Weisz, Joseph Melber, Yu Wang, Kermin Fleming, Eriko Nurvitadhi, James C. Hoe, A study of pointer-chasing performance on shared-memory processor-FPGA systems, in: Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2016, pp. 264–273.
- [24] Onur Kocberber, Babak Falsafi, Boris Grot, Asynchronous memory access chaining, Proc. VLDB Endow. 9 (4) (2015) 252–263.
- [25] Mustafa Cavus, Resit Sendag, Joshua J. Yi, Informed prefetching for indirect memory accesses, ACM Trans. Archit. Code Optim. (TACO) 17 (1) (2020) 1–29.
- [26] Ricardo Alves, Stefanos Kaxiras, David Black-Schaffer, Early address prediction: Efficient pipeline prefetch and reuse, ACM Trans. Archit. Code Optim. (TACO) 18 (3) (2021) 1–22.
- [27] Hikaru Takayashiki, Masayuki Sato, Kazuhiko Komatsu, Hiroaki Kobayashi, A hardware prefetching mechanism for vector gather instructions, in: 2019 IEEE/ACM 9th Workshop on Irregular Applications: Architectures and Algorithms, IA3, IEEE, 2019, pp. 59–66.
- [28] Sam Ainsworth, Timothy M. Jones, Software prefetching for indirect memory accesses: A microarchitectural perspective, ACM Trans. Comput. Syst. (TOCS) 36 (3) (2019) 1–34.
- [29] Kevin Hsieh, Samira Khan, Nandita Vijaykumar, Kevin K Chang, Amirali Boroumand, Saugata Ghose, Onur Mutlu, Accelerating pointer chasing in 3D-stacked memory: Challenges, mechanisms, evaluation, in: 2016 IEEE 34th International Conference on Computer Design, ICCD, IEEE, 2016, pp. 25–32.
- [30] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, Russell Sears, Benchmarking cloud serving systems with YCSB, in: Proceedings of the 1st ACM Symposium on Cloud Computing, 2010, pp. 143–154.
- [31] Thomas Neumann, Tobias Mühlbauer, Alfons Kemper, Fast serializable multi-version concurrency control for main-memory database systems, in: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, 2015, pp. 677–689.
- [32] Yu Xia, Xiangyao Yu, Andrew Pavlo, Srinivas Devadas, Taurus: Lightweight Parallel Logging for In-Memory Database Management Systems, VLDB Endowment, 2020.
- [33] Takayuki Tanabe, Takashi Hoshino, Hideyuki Kawashima, Jun Nemoto, Masahiro Tanaka, Osamu Tatebe, An analysis of concurrency control protocols for in-memory databases with CCBench (extended version), 2020, arXiv preprint arXiv:2009.11558.
- [34] Amir Roth, Gurindar S. Sohi, Effective jump-pointer prefetching for linked data structures, in: Proceedings of the 26th Annual International Symposium on Computer Architecture, 1999, pp. 111–121.
- [35] Shimin Chen, Redesigning Database Systems in Light of CPU Cache Prefetching (Ph.D. thesis), Carnegie Mellon University, Pittsburgh, PA, USA, 2005.
- [36] Arlino Magalhaes, Jose Maria Monteiro, Angelo Brayner, Main memory database recovery: A survey, ACM Comput. Surv. 54 (2) (2021) 1–36.
- [37] Christie L. Alappat, Johannes Hofmann, Georg Hager, Holger Fehske, Alan R. Bishop, Gerhard Wellein, Understanding HPC benchmark performance on intel broadwell and cascade lake processors, in: High Performance Computing: 35th International Conference, ISC High Performance 2020, Frankfurt/Main, Germany, June 22–25, 2020, Proceedings 35, Springer, 2020, pp. 412–433.
- [38] Miles Dai, Reverse Engineering the Intel Cascade Lake MeshInterconnect (Ph.D. thesis), Massachusetts Institute of Technology, 2021.
- [39] Shuo Li, Zhiguang Chen, Nong Xiao, Guangyu Sun, Path prefetching: Accelerating index searches for in-memory databases, in: 2018 IEEE 36th International Conference on Computer Design, ICCD, IEEE, 2018, pp. 274–277.
- [40] Anastasia Ailamaki, David J. DeWitt, Mark D. Hill, Data page layouts for relational databases on deep memory hierarchies, VLDB J. 11 (2002) 198–215.
- [41] Richard A. Hankins, Jignesh M. Patel, Data morphing: An adaptive, cache-conscious storage technique, in: Proceedings 2003 VLDB Conference, Elsevier, 2003, pp. 417–428.
- [42] Shimin Chen, Phillip B. Gibbons, Todd C. Mowry, Improving index performance through prefetching, ACM SIGMOD Rec. 30 (2) (2001) 235–246.
- [43] SkyLake. <https://www.7-cpu.com/cpu/Skylake.html>.
- [44] CoffeeLake, 2017, https://en.wikichip.org/wiki/intel/microarchitectures/coffee_lake.
- [45] CascadeLake, 2019, https://en.wikichip.org/wiki/intel/microarchitectures/cascade_lake.
- [46] Intel memory latency checker v3.10, 2021, <https://www.intel.com/content/www/us/en/developer/articles/tool/intelr-memory-latency-checker.html>.
- [47] Martin Grund, Philippe Cudré-Mauroux, Jens Krüger, Samuel Madden, Hasso Plattner, An overview of HYRISE—a main memory hybrid storage engine, IEEE Data Eng. Bull. 35 (1) (2012) 52–57.
- [48] Adam Prout, Szu-Po Wang, Joseph Victor, Zhou Sun, Yongzhu Li, Jack Chen, Evan Bergeron, Eric Hanson, Robert Walzer, Rodrigo Gomes, et al., Cloud-native transactions and analytics in SingleStore, in: Proceedings of the 2022 International Conference on Management of Data, 2022, pp. 2340–2352.

- [49] Ji-Tae Yun, Su-Kyung Yoon, Jeong-Geun Kim, Shin-Dug Kim, Access pattern-based high-performance main memory system for graph processing on single machines, *Future Gener. Comput. Syst.* 108 (2020) 560–573.
- [50] Jan Böttcher, Viktor Leis, Thomas Neumann, Alfons Kemper, Scalable garbage collection for in-memory MVCC systems, *Proc. VLDB Endow.* 13 (2) (2019) 128–141.
- [51] Jongbin Kim, Kihwang Kim, Hyunsoo Cho, Jaeseon Yu, Sooyong Kang, Hyungsoo Jung, Rethink the scan in MVCC databases, in: *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 938–950.
- [52] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, et al., Manu: a cloud native vector database management system, 2022, arXiv preprint [arXiv:2206.13843](https://arxiv.org/abs/2206.13843).
- [53] Timothy King. 2019, 80 Percent of your data will be unstructured in five years, 2019, <https://solutionsreview.com/data-management/80-percent-of-your-data-will-be-unstructured-in-five-years/>.



Qian Zhang received a B.S. and M.S. degrees in Computer Science and Information Engineering from HeNan University, China, in 2012 and 2014. She is currently pursuing a Ph.D. degree in the Software Engineering Institute at East China Normal University, China. From 2014 to 2018, she was a software development engineer with the financial technology company of Hundsun, headquartered in Hangzhou, China. Her research interests include in-memory database systems, distributed database systems, computer systems, data structure, memory access optimization, and machine learning with databases.



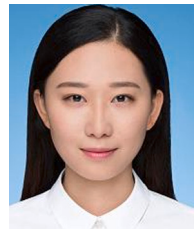
Haoyun Song received a B.S. degree from Sichuan University in 2022. She is currently pursuing an M.S. degree from the Software Engineering Institute, East China Normal University, China and an M.B.A. degree from the University of Haifa, Israel. Her research interests include distributed databases, in-memory database systems, and query optimization techniques.



Kaiyan Zhou received a Bachelor's degree from Sichuan University and is currently studying at East China Normal University. His current research focuses on in-memory databases and distributed database systems. He is particularly interested in issues within the database kernel domain and has a substantial reserve of relevant knowledge.



Jianhao Wei received an M.S. degree from the Software Engineering Institute, East China Normal University, China, where he is currently pursuing a Ph.D. degree. He has published one journal paper in the *Journal of Software*. His main research interests include multi-Query sharing technology, in-memory database systems, persistent hardware, query optimization techniques, and machine learning with databases.



Chuqiao Xiao received a B.S. degree from the Tianjin Polytechnic University of computer science and technology in 2015 and an M.S degree in computer technology from East China Normal University in 2018, where he is currently pursuing a Ph.D. degree in the software engineering institute. Her research interests include distributed databases, key-value storage systems, in-memory database systems, and erasure codes.